

PROJECT REPORT

Project Name	ResQnet	Reporting Period
Project Owner	Viren Singh	Jul 24, 2026 to Jul 30, 2026
Prepared by	Viren Singh	

HIGHLIGHTS

- ResQNet is a project I started because I always found disaster management and rescue systems really interesting, specially how technology can actually help people during emergencies.
- I made a map based system where dangerous areas and safer routes can be viewed properly so rescue planning becomes easier and more organized.
- I tried using algorithms like Dijkstra for route finding and spent a lot of time fixing and testing how the paths should work between different locations.
- Most of the project was made by me using JavaScript and web technologies, where I mainly worked on the maps, frontend and connecting all the different parts together.
- I also experimented with some small features related to emergency alerts and coordination between rescue points so the system could feel more practical.
- While building this project I learned many things about debugging, optimization and how difficult real world systems actually are.
- I kept improving the project in my free time during class 11th and 12th because I genuinely enjoyed working on it.

CHALLENGES

- One of the hardest parts while making ResQNet was getting everything to work together properly because many times the maps, routes or alerts used to break randomly and I had to spend hours figuring out what went wrong.
- I also had problems while working on the route finding system because sometimes the rescue paths were not updating correctly between locations on the map.

Live project : link

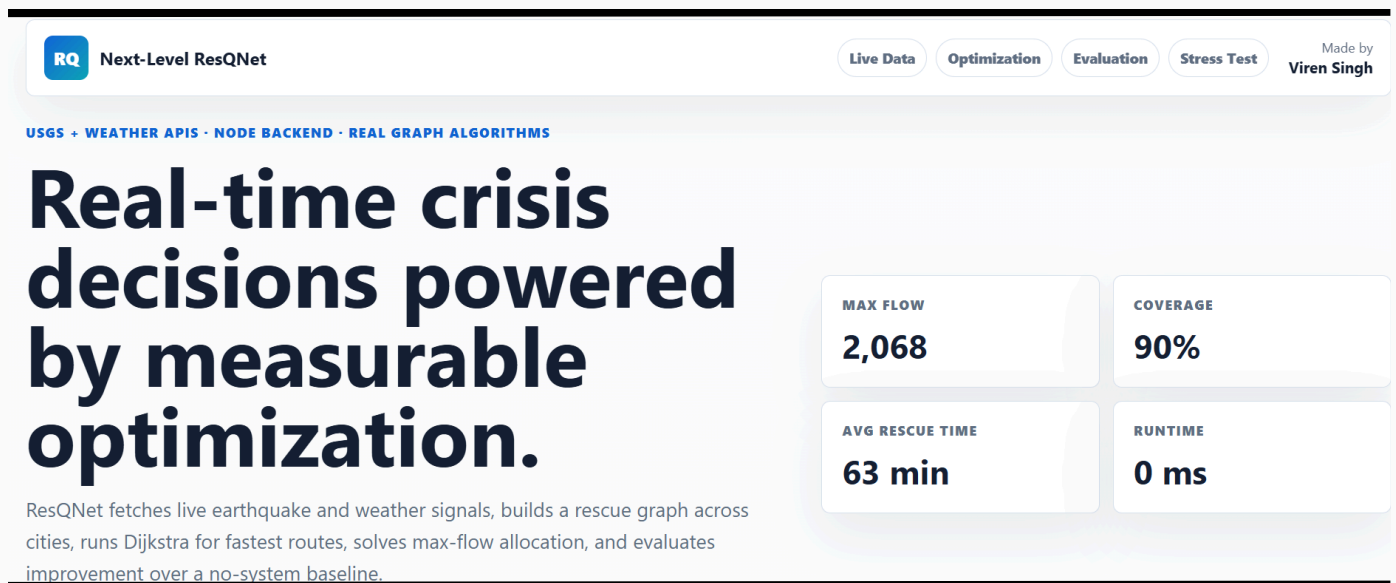
<https://virensingh011.github.io/ResQnet/>

Github Repository:

<https://github.com/virensingh011>

System showcase:- screenshot

Dashboard



LIVE DATA, NO GUESSING –

Pulls real earthquake and weather info so decisions aren't based on old news.

• **A RESCUE MAP THAT CHANGES WITH REALITY** –

Cities connect based on actual conditions, not just distance.

- **FINDS THE FASTEST ROUTE,**
EVERY TIME – Uses Dijkstra’s algorithm to cut down rescue travel minutes.
-
- **SMART RESOURCE FLOW** –
Solves max-flow to send exactly the right amount of help where it’s needed.
-
- **PROOF IT WORKS** – Compares results against doing nothing, so you can see the real improvement.
- **ONE-CLICK TESTING** – Run the system normally or stress-test it to see how tough it really is.

Mission control dashboard

Live Incidents					4 incidents	
INCIDENT	CITY	SOURCE	PEOPLE	SEVERITY		
M6.4 Bay Area earthquake	San Francisco	Static USGS-style fallback	1,850	91%		
Los Angeles seismic corridor	Los Angeles	Static USGS-style fallback	2,300	88%		
Miami severe weather response	Miami	Static weather fallback	1,500	82%		
Delhi flood risk corridor	Delhi	Static weather fallback	2,600	79%		

Optimized Rescue Plan		12 nodes · 8 edges	
1	Miami Emergency Operations → Miami severe weather response 745 people-equivalent capacity, 24 min route, 100% edge utilization.	Bottleneck	
2	AIIMS Emergency Hub → Delhi flood risk corridor 1,545 people-equivalent capacity, 33 min route, 100% edge utilization.	Bottleneck	
3	Oakland Logistics Base → M6.4 Bay Area earthquake 809 people-equivalent capacity, 35 min route, 100% edge utilization.	Bottleneck	
4	LA County Medical Hub → Los Angeles seismic corridor 1,228 people-equivalent capacity, 39 min route, 100% edge utilization.	Bottleneck	
5	Dade Shelter Network → Miami severe weather response 719 people-equivalent capacity, 45 min route, 100% edge utilization.	Bottleneck	
6	SF General Hospital → M6.4 Bay Area earthquake 789 people-equivalent capacity, 49 min route, 100% edge utilization.	Bottleneck	
7	NCR Logistics Depot → Delhi flood risk corridor 1,055 people-equivalent capacity, 69 min route, 82% edge utilization.	Optimal	
8	Long Beach Port Logistics → Los Angeles seismic corridor 840 people-equivalent capacity, 97 min route, 100% edge utilization.	Bottleneck	

LIVE INCIDENTS – just the current situation. Earthquake in San Francisco, flood in Delhi, storm in Miami. Each row shows the city, how many people are affected, and the severity percentage. No interpretation yet, just data.

OPTIMIZED RESCUE PLAN – this is where the system does the work. It matches each incident to a specific rescue hub, then tells you:

- **CAPACITY** (how many people that hub can realistically handle)

- **ROUTE TIME** (minutes)
- **EDGE UTILIZATION** (how crowded that route is)

WHY IT IS USEFUL

Because you don't want to open five different tools during an active incident. This page gives you the assignment and the constraint in one place. You see what's working and what's about to break without having to dig

Before & After: resQnet impact

Try Rutherford, Bohr, thermodynamics, Maxwell...			All domains
Evaluation: Without System vs With ResQNet			+34% coverage - 40 min saved
SCENARIO	WITHOUT SYSTEM	WITH RESQNET	
Rescue time	88 min	48 min	
Coverage	60%	94%	
Unmet demand	3303 people	520 people	
Full operation makespan	244 min	153 min	
Bottleneck awareness	Not measured	7 routes flagged	

WHAT THIS FEATURE DOES

A direct before-and-after comparison of the same incident set.

Left column – performance without the system.

Right column – performance with ResQNet.

Key metrics shown side by side: rescue time, coverage, unmet demand, makespan.

The bottom row automatically calculates the improvement — in this case, +34% coverage and 40 minutes saved.

No charts. No extra clicks. Just a clear answer to: *"Does*

this actually work?"

Scaling & Stress Test

Scaling + Stress Test		PASS
CITIES 40	INCIDENTS 40	
GRAPH 80 nodes · 282 edges	RUNTIME 34 ms	
MAX FLOW 21,440	COVERAGE 78%	

WHAT THIS FEATURE DOES

It's a stress test view for the system.

Instead of simulating one incident or one city, it scales up — in this case to 40 incidents across 40 cities.

What it shows you in one place:

- **PASS/FAIL STATUS** – at the top. Immediate confidence check.
- **SCALE** – how many cities and incidents were tested.
- **SYSTEM PERFORMANCE** – runtime in

milliseconds, graph size (nodes and edges).

- **OUTPUT METRICS** – max flow and coverage percentage under load.

Why this feature exists

Because a system that works for one incident might break at scale. This view tells you: *"We pushed it hard, and here's how it held up."*

No guesswork. Just a clean summary of whether the system is production-ready.

System Architecture

System Architecture

Backend data ingestion

Node.js fetches USGS earthquake GeoJSON and Open-Meteo weather data.

Dijkstra implementation

Server builds graph nodes and edges, then computes shortest hub-to-crisis routes.

Edmonds-Karp max-flow

Allocates finite rescue capacity through constrained graph edges.

Evaluation + stress

Compares baseline response against ResQNet and tests scaling with synthetic cities/incidents.

WHAT THIS FEATURE DOES

It shows you how the system is built, plain and simple.

Four layers, top to bottom:

- 1. BACKEND DATA INGESTION** – what data goes in and where it comes from (USGS earthquakes, Open-Meteo weather).
- 2. DIJKSTRA IMPLEMENTATION** – how the system finds the shortest routes between hubs and crises.
- 3. EDMONDS-KARP MAX-FLOW** – how it allocates limited rescue capacity across the network.
- 4. EVALUATION + STRESS** – how it tests

itself against baselines and at scale.

Why this feature exists

Because you shouldn't have to guess how a system works.

This page gives any stakeholder — technical or not — a clear, honest map of the logic under the hood.

No diagrams. No jargon dump. Just what each piece does.